



AUTOPART-SYNCDISTRIBUTED · PROTÓTIPO ACADÊMICO LOCAL · 2026

Diagrama de Arquitetura de Software

Do navegador ao MongoDB: componentes, integrações e responsabilidades do protótipo local.

VISÃO TÉCNICA · FLUXO DE DADOS E DEPENDÊNCIAS INTERNAS



Aplicação acadêmica local · Não representa múltiplos servidores ou alta disponibilidade

Três camadas na mesma demonstração local · Sem pagamento real ou alta disponibilidade.

MOTIVAÇÃO

Problema e valor proposto

O problema

Estoque controlado à mão: anotações lentas, erros de contagem, preço inconsistente e **nenhum histórico confiável de pedidos**.

O valor proposto

Catálogo digital com busca e filtros, carrinho e pedidos persistentes, e **estoque centralizado no servidor** como fonte única de verdade.

Tudo em execução local — protótipo destinado à demonstração acadêmica, sem operação comercial.

QUEM USA

Usuários e papéis

Cliente final

Explora o catálogo, busca peças compatíveis com o veículo, monta o carrinho, finaliza o pedido (*checkout simulado*) e acompanha seu histórico.

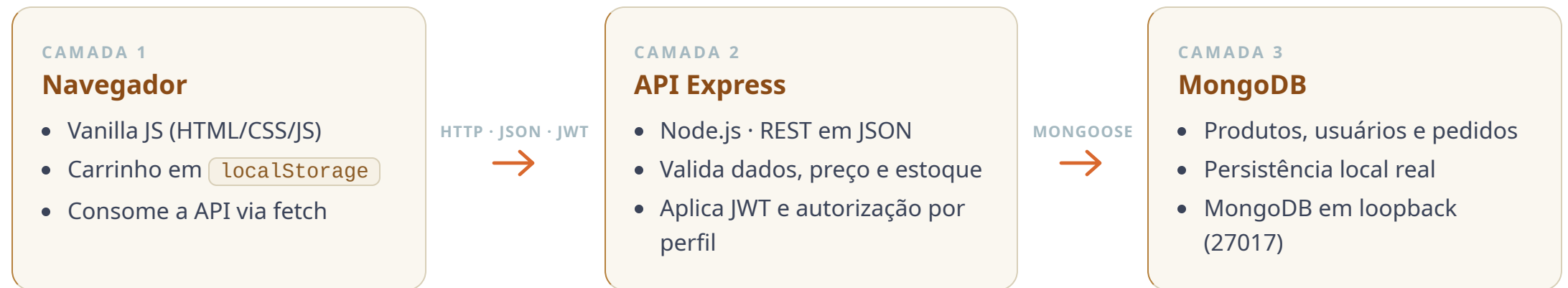
Administrador

Cadastra e edita peças, preço e estoque, acompanha os pedidos e atualiza seus status — *sempre via API protegida* por perfil.

Contas genéricas da demo, se o seed local estiver habilitado: `admin.demo@autopart.test` · `cliente.demo@autopart.test`. A senha é definida localmente, não publicada aqui.

ARQUITETURA

Navegador → API Express → MongoDB



O servidor é a autoridade: preço, estoque e pedido são decididos e revalidados no back-end — nunca somente no navegador.

FRENTE DE LOJA

Vitrine, catálogo e busca

- ◆ **Vitrine pública e catálogo** em `GET /api/catalogo` — consulta sem login.
- ◆ **Busca por nome, marca e modelo**, com filtros por categoria e disponibilidade.
- ◆ **Ficha da peça**: compatibilidade (marca/modelo), preço e estoque visível.
- ◆ Navegação direta para carrinho, conta e — com login administrador — a área administrativa.

FLUXO DE COMPRA

Carrinho, checkout e histórico

- 1 Carrinho no navegador — itens e quantidades em `localStorage`.
- 2 Checkout autenticado — `POST /api/orders/checkout` revalida itens, preços e estoque no servidor.
- 3 Pedido persistente — gravado no MongoDB com status próprio.
- 4 Histórico do cliente — `GET /api/orders/me` e `GET /api/orders/:id`.

⚠ Checkout simulado — sem pagamento real.

OPERAÇÃO

Administração e inventário

- ◆ **Área administrativa** atrás do prefixo protegido `/api/admin`.
- ◆ **Criação e edição de peças:** nome, compatibilidade, preço e estoque.
- ◆ **Pedidos:** acompanhamento e atualização de status.
- ◆ Consulta e manutenção apoiadas na **autorização por perfil no servidor**.

SEGURANÇA

Segurança e autorização

- ◆ **Senhas com hash** `bcryptjs` — nunca armazenadas nem devolvidas em texto puro.
- ◆ **Sessões JWT assinadas** — `JWT_SECRET` privado, longo e fornecido por configuração; nunca versionado.
- ◆ **Autorização por perfil verificada no servidor** — não apenas ocultada na interface.

- **Verificado localmente: cliente autenticado recebe 403 no endpoint administrativo**

QUALIDADE VERIFICADA

Evidências de teste e execução

22/22

testes automatizados aprovados nas Fases 1-6 · 0
falhas · verificação de **16/09/2026**

- ✓ Home e catálogo respondem 200
- ✓ Login de admin e de cliente com a senha de demonstração configurada
- ✓ Pedidos do cliente autenticado: 200
- ✓ Produtos e pedidos do administrador: 200
- ✓ Cliente recebe 403 no endpoint administrativo

Evidência de execução local; não representa aceitação final nem ambiente público.

ESCOPO HONESTO

Limites do protótipo

- ◆ **Checkout simulado:** não há pagamento, entrega, logística ou integração externa.
- ◆ **Execução local acadêmica:** sem deploy público, sem URL pública e sem SLA.
- ◆ **Sem suíte Playwright E2E:** screenshots locais reais cobrem vitrine, administração e este deck; não são seis provas visuais separadas.
- ◆ **Protótipo de estudo:** reutilização exige nova avaliação de segurança, privacidade e operação.

ANTES DE APRESENTAR

Demonstração local — sequência de verificação

- 1 Subir o banco: `docker compose up -d mongodb`
- 2 Preparar `.env` a partir de `.env.example`: `JWT_SECRET` privado (32+ caracteres); para contas demo, ativar `SEED_DEMO_DATA=true` e definir `SEED_DEMO_PASSWORD` localmente.
- 3 Iniciar o servidor: `node server/index.js`
- 4 Abrir `http://127.0.0.1:3000` (ou usar a porta configurada em `PORT`, por exemplo 5500).
- 5 Se o seed foi habilitado, entrar como `admin.demo@autopart.test` e `cliente.demo@autopart.test` com a senha definida localmente.
- 6 Checklist rápido: catálogo 200 · pedido do cliente · produtos e pedidos do admin · 403 do cliente em `/api/admin`

Isto é uma checagem pré-apresentação; o runtime local não é alegado como permanentemente online. Nunca exibir o valor de `JWT_SECRET`.

FECHAMENTO

Conclusão e repositório

O que ficou demonstrado

Separação real de camadas, autorização no servidor, checkout simulado declarado como tal e **22/22 testes verificados** — um protótipo local honesto.

Repositório GitHub

Código, README com execução local verificada e documentação no repositório do projeto **AutoPart-SyncDistributed** — base reaproveitável para trabalhos futuros após nova avaliação.

Obrigado! Perguntas?